



Nexus SMARTS/SMARTER

Manual do Programador & Especificações de Resolvedores

Organização: NEXUS-MCDM

Status: Distribuição Geral

Ambiente: Integrado / Standalone

Copyright: Todos os direitos reservados à NEXUS-MCDM.

1. Arquitetura de Software do Hub

O ecossistema Nexus MCDM é estruturado com uma separação clara entre a interface do usuário (Flask/Jinja2) e os motores de cálculo matemáticos localizados em modules/. Cada resolvidor é implementado em Python utilizando estruturas vetoriais do NumPy e solucionadores de Programação Linear.

2. Pseudocódigos dos Solucionadores MCDM

Abaixo estão listados os pseudocódigos fundamentais dos principais resolvidores do sistema:

ALGORITMO 1: TOPSIS Engine

```

1: Receber Matriz X (m x n), pesos W, e critérios de custo/benefício C_types
2: Para cada coluna j = 1 a n:
3: Denom = raiz_quadrada( soma( x_ij^2 ) para i = 1 a m )
4: Para cada linha i = 1 a m:
5: r_ij = x_ij / Denom
6: v_ij = r_ij * w_j
7: Identificar Ideal Positivo (A+) e Negativo (A-):
8: Se C_types[j] == 'benefit': A_j+ = max(v_j), A_j- = min(v_j)
9: Se C_types[j] == 'cost': A_j+ = min(v_j), A_j- = max(v_j)
10: Para cada alternativa i = 1 a m:
11: S_i+ = raiz_quadrada( soma( (v_ij - A_j+)^2 ) para j = 1 a n )
12: S_i- = raiz_quadrada( soma( (v_ij - A_j-)^2 ) para j = 1 a n )
13: C_i = S_i- / (S_i+ + S_i-)
14: Retornar ranking ordenado decrescente baseado em C_i

```

ALGORITMO 2: VIKOR Engine

```

1: Receber Matriz X, pesos W, critério de custo/benefício, e parâmetro v
2: Determinar melhores (f_j*) e piores (f_j-) valores físicos por critério
3: Para cada alternativa i = 1 a m:
4: S_i = soma( w_j * (f_j* - x_ij) / (f_j* - f_j-) ) para j = 1 a n
5: R_i = max( w_j * (f_j* - x_ij) / (f_j* - f_j-) ) para j = 1 a n
6: Determinar S* = min(S_i), S- = max(S_i), R* = min(R_i), R- = max(R_i)
7: Para cada alternativa i = 1 a m:
8: Q_i = v * (S_i - S*) / (S- - S*) + (1-v) * (R_i - R*) / (R- - R*)
9: Retornar ranking ordenado crescente baseado em Q_i

```

ALGORITMO 3: ELECTRE TRI Category Sorting

```

1: Receber Perfis Limites b_1, b_2, ..., b_k, pesos W, e limiares (q, p, v)
2: Para cada alternativa a_i e perfil b_h:
3: Calcular Concordância c(a_i, b_h) agregada por pesos
4: Calcular Discordância local d_j(a_i, b_h) e verificar limiar de Veto
5: Determinar Credibilidade rho(a_i, b_h) incorporando concordância e vetos
6: Relação de Sobreclassificação: a_i S b_h se rho(a_i, b_h) >= lambda
7: Alocação Pessimista:
8: Encontrar o maior perfil b_h (de k a 1) onde a_i S b_h
9: Alocar a_i para a categoria h + 1 (Categoria 1 é a pior)

```

ALGORITMO 4: PROMETHEE V Portfolio Optimization

- 1: Receber Fluxos Líquidos $Net_Phi(a_i)$, Custos c_i , e Orçamento limite B
- 2: Definir Variáveis de Decisão Binárias: x_i em $\{0, 1\}$
- 3: Maximizar a Função Objetivo: $Z = soma(Net_Phi(a_i) * x_i)$
- 4: Sujeito a Restrição Orçamentária: $soma(c_i * x_i) \leq B$
- 5: Resolver via Otimização de Mochila Inteira (0-1 Knapsack)
- 6: Retornar projetos selecionados ($x_i = 1$)